

Titolo progetto	DLine
Studiante	Battaglia Daniel
Periodo	Marzo – Maggio 2026
Scopo	Progetto personale, non accademico, svolto per acquisire esperienza pratica nel campo IoT. Progettare un dispositivo hardware e programmarlo per eseguire azioni specifiche. Le pagine web (sia client che server) sono scritte in PHP, mentre il firmware dell'hardware (caricato dentro Arduino ESP32 Nano) in C++.

Sommario

1. Introduzione e Analisi del Problema	2
1.1. Descrizione del Progetto	2
1.2 Obiettivi del Sistema	2
1.3 Analisi del Sistema Attuale	2
1.4 Sistema Proposto	3
2. Architettura del Sistema	3
2.1 Panoramica	3
2.2 Infrastruttura Hardware	3
2.3 Stack tecnologico software	4
2.2 UML Deployment Diagram	4
3. Implementazione Hardware	5
3.1 Scelte Progettuali	5
3.2 Elenco Componenti e Specifiche Tecniche	5
3.2.1 Microcontrollore ESP32	5
3.2.2 Modulo Display OLED SSD1306	6
3.2.3 Stampante Termica EM5822	7
3.2.4 Pulsante	8
3.2.5 Sistema di Alimentazione	8
3.2.6 Breadboard & cavi	9
3.3 Schema di Collegamento	9
4. Sviluppo Software	10
4.1 Panoramica	10
4.2 Ambienti di Sviluppo e Testing Locale	10
4.3 Protocolli di Comunicazione e Tecnologie Web	10
4.3 Logica e Funzionamento del Firmware	10
4.4 Persistenza dei Dati (Database MySQL)	11
4.5 Integrazione API e Sincronizzazione di Rete	11
5. Interfaccia (Mockup)	12
5.1 Interfacce Web: Frontend Pubblico e Admin	12

5.2 Progettazione del Contenitore Fisico (Case Design).....	13
6. Analisi Economica	14
6.1 Distinta Base (BOM)	14
6.2 Costi infrastrutturali.....	14
6.2 Valutazione totale	14
7. Conclusioni e Sviluppi Futuri	15
7.1 Trade Off	15
7.1 Risultati Ottenuti	15
7.2 Possibili Implementazioni.....	15
8. Glossario	16

1. Introduzione e Analisi del Problema

1.1. Descrizione del Progetto

Il progetto riguarda lo sviluppo di **DLine**, un sistema **IoT** per la gestione delle code all'interno di attività commerciali. Sebbene il concetto di eliminacode sia consolidato, l'iniziativa nasce come applicazione pratica di competenze multidisciplinari, con l'obiettivo di realizzare un'architettura **client-server** integrata. Il funzionamento si basa sull'interazione tra un dispositivo fisico (il Client) e un'infrastruttura **Web Hosting** (il Server). Attraverso il dispositivo DLine, l'utente richiede un ticket premendo un pulsante; l'azione viene registrata dal web hosting e il risultato viene contestualmente visualizzato su suo display e stampato tramite una stampante termica. Il sistema si completa con un'interfaccia web dedicata: una pagina web che mostra il numero in fase di servizio e il totale delle persone in attesa; e una sezione riservata al personale (es. cassieri) per gestire l'avanzamento dei numeri o il reset del sistema.

L'idea è nata durante la mia esperienza lavorativa da Vanessa Sound, che usava dispositivi simili, stimolando il mio desiderio di approfondirne il funzionamento logico e tecnico attraverso la creazione di un prototipo funzionante.

1.2 Obiettivi del Sistema

L'obiettivo primario di **DLine** è l'ottimizzazione del flusso di utenza all'interno di contesti commerciali o di servizio. Nello specifico, il progetto mira a:

- **Digitalizzare il processo di attesa:** eliminando la confusione visiva e verbale all'interno del locale.
- **Garantire la sincronizzazione remota:** permettendo la visualizzazione dei numeri in coda su diversi dispositivi (monitor, smartphone, tablet) simultaneamente.
- **Fornire uno strumento di controllo agile:** dare al personale (admin) la possibilità di gestire la coda con un semplice click da qualsiasi postazione web.
- **Ridurre l'errore umano:** automatizzando l'incremento dei numeri e la stampa dei ticket.

1.3 Analisi del Sistema Attuale

Attualmente, molte piccole e medie attività gestiscono le code attraverso metodi che presentano diverse criticità:

- **Sistemi Manuali/Verbali:** si basano sulla memoria dei clienti o sul richiamo vocale del personale. Questo genera stress, dispute tra i clienti e interruzioni costanti del lavoro dei dipendenti.

- **Sistemi Meccanici "Chiocciola":** pur fornendo un ticket cartaceo, non offrono alcuna sincronizzazione digitale. Se il cliente si allontana, non ha modo di sapere a che punto è la fila; inoltre, il personale deve aggiornare manualmente display analogici che spesso non sono visibili da ogni angolo del locale.

1.4 Sistema Proposto

DLine si propone come una soluzione intermedia tra i costosi sistemi eliminacode industriali e la gestione manuale, offrendo i vantaggi tipici dell'IoT:

- **Automazione Integrata:** con un solo gesto (pressione del pulsante), il sistema gestisce contemporaneamente database, stampa e visualizzazione.
- **Accessibilità Multiplatforma:** grazie all'architettura Web, il monitor della coda può essere proiettato su una TV, visto su un PC o consultato tramite tablet, senza necessità di cablaggi video complessi.
- **User Experience Migliorata:** il cliente riceve un ticket chiaro e può monitorare l'avanzamento su display ad alta risoluzione, riducendo la percezione del tempo di attesa.

2. Architettura del Sistema

2.1 Panoramica

L'architettura di si basa su un modello **Client-Server**, concepito per garantire la persistenza dei dati e una sincronizzazione in tempo reale tra il mondo fisico e le interfacce digitali:

- **Client (Dispositivi e Interfacce):** Rappresenta i punti di interazione con l'utente e si divide in due tipologie:
 - **Client Hardware:** Il dispositivo fisico DLine, basato sul microcontrollore ESP32. È incaricato di gestire gli eventi fisici (come la pressione del pulsante), produrre l'output visivo locale (display), gestire la stampa dei ticket e comunicare via rete con il server.
 - **Client Web:** I dispositivi (smartphone, tablet, PC) che, tramite browser, accedono alle pagine web dedicate alla visualizzazione pubblica del numero o al pannello di controllo amministrativo.
- **Server (Web Hosting):** Rappresenta il "cervello" centrale dell'infrastruttura. Ospita il database e le risorse software necessarie. Riceve i dati dai client tramite chiamate API, elabora e aggiorna lo stato della coda, e restituisce le informazioni aggiornate a tutti i dispositivi connessi.

2.2 Infrastruttura Hardware

Per la realizzazione del dispositivo fisico, il sistema si avvale di una serie di moduli elettronici selezionati per ottimizzare costi e prestazioni. Il coordinamento è affidato al **Microcontrollore ESP32**, a cui sono collegati i seguenti elementi periferici:

- **Modulo display** (per il feedback visivo sul dispositivo).
- **Pulsante** (per l'input dell'utente).
- **Stampante termica** (per l'erogazione del ticket cartaceo ed alimentata separatamente).
- **Breadboard e cavi** (per il cablaggio e l'interconnessione logica).

Una volta cablati, tutti i componenti sono assemblati e protetti all'interno di un **contenitore dedicato, realizzato tramite stampa 3D**, progettato per compattare l'hardware e rendere il prodotto idoneo all'esposizione su un bancone.

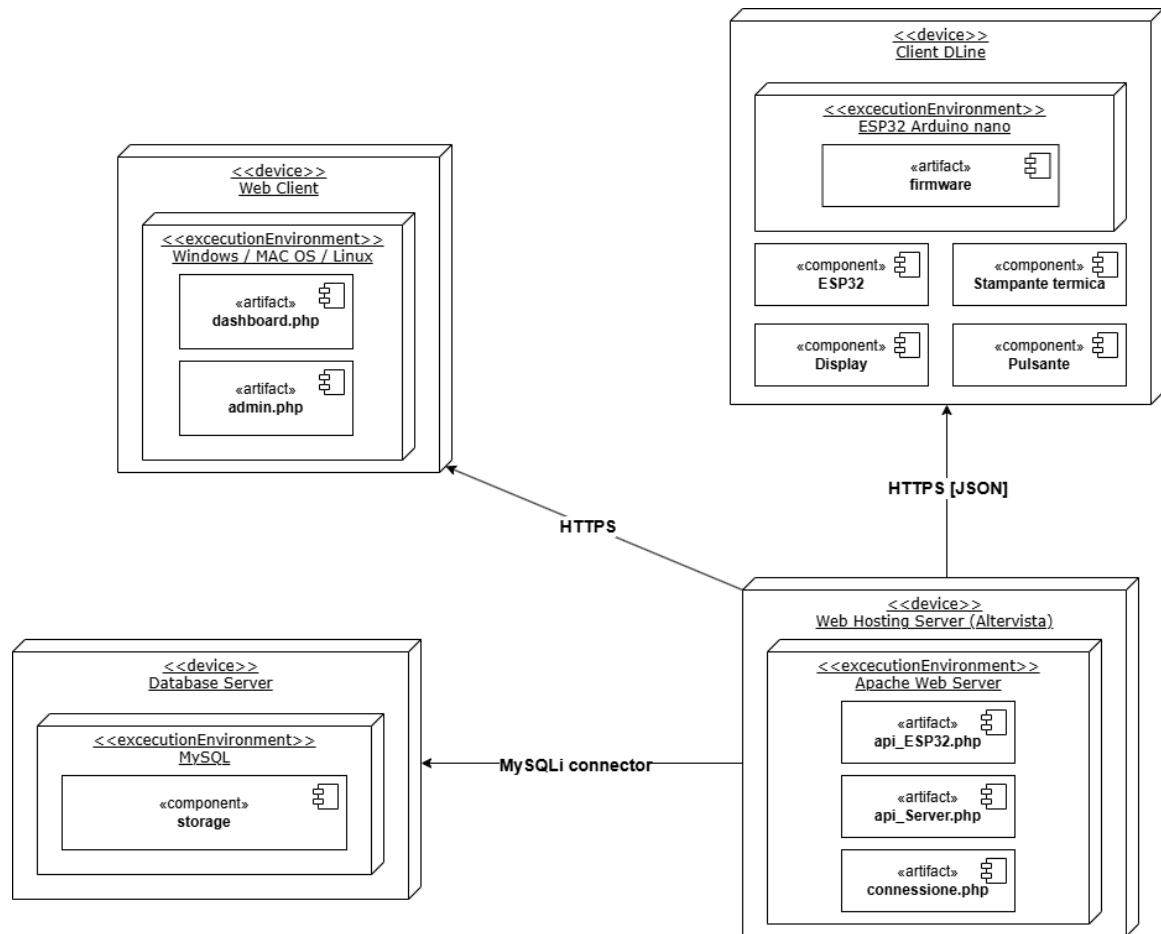
2.3 Stack tecnologico software

Il corretto funzionamento del sistema e la comunicazione fluida tra l'hardware e il server richiedono l'impiego di diverse tecnologie. Lo sviluppo è stato suddiviso in base al livello di competenza, utilizzando gli strumenti e i linguaggi riepilogati nella seguente tabella:

IDE	FIRMWARE(Client)	BACK END	FRONT END
Arduino IDE, Visual Studio Code	C++	HTML, CSS, JavaScript	PHP, SQL,

2.2 UML Deployment Diagram

In questa sezione viene mostrato il **Deployment Diagram**, che serve a illustrare l'architettura fisica del sistema **DLine**. Questo schema permette di visualizzare in modo immediato l'hardware coinvolto (il dispositivo IoT, il server web e i client utente), dove risiedono fisicamente i vari moduli software e attraverso quali protocolli di comunicazione e di rete interagiscono tra loro.



3. Implementazione Hardware

3.1 Scelte Progettuali

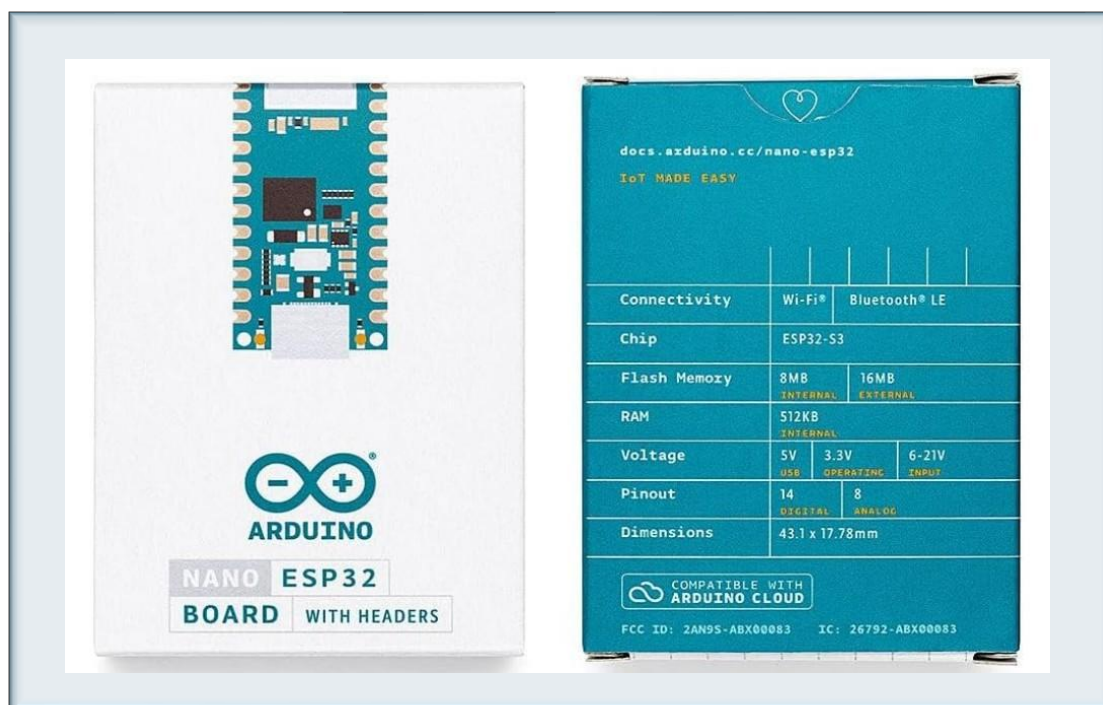
La fase di progettazione hardware è stata guidata dalla necessità di coniugare affidabilità industriale, dimensioni contenute e costi di produzione ridotti. Le scelte principali che hanno definito l'architettura fisica del dispositivo sono le seguenti:

- **Selezione del Microcontrollore:** La scelta è ricaduta sull'**ESP32** rispetto a piattaforme più tradizionali (come Arduino Uno o Nano) per la presenza nativa del modulo Wi-Fi e Bluetooth. Questo ha permesso di evitare l'aggiunta di shield esterne, riducendo l'ingombro del circuito e semplificando la gestione del protocollo HTTPS necessario per la comunicazione con il web hosting. Inoltre, la potenza di calcolo superiore (Dual Core) garantisce una gestione fluida delle code di stampa e degli aggiornamenti del display in parallelo.
- **Gestione dell'Alimentazione:** Data la presenza di una stampante termica, che presenta picchi di assorbimento di corrente elevati durante la fase di stampa, si è scelto di separare idealmente le linee di alimentazione o di utilizzare un alimentatore ad alto amperaggio (almeno 2A) per evitare cali di tensione che potrebbero causare il riavvio (brownout) del microcontrollore.
- **Modularità e Form Factor:** Il design è stato pensato per essere compatto, adatto a essere posizionato su banconi di piccole dimensioni. La scelta di componenti modulari (display e stampante separati dal core) facilita la manutenzione e la sostituzione dei singoli elementi in caso di guasto.

3.2 Elenco Componenti e Specifiche Tecniche

3.2.1 Microcontrollore ESP32

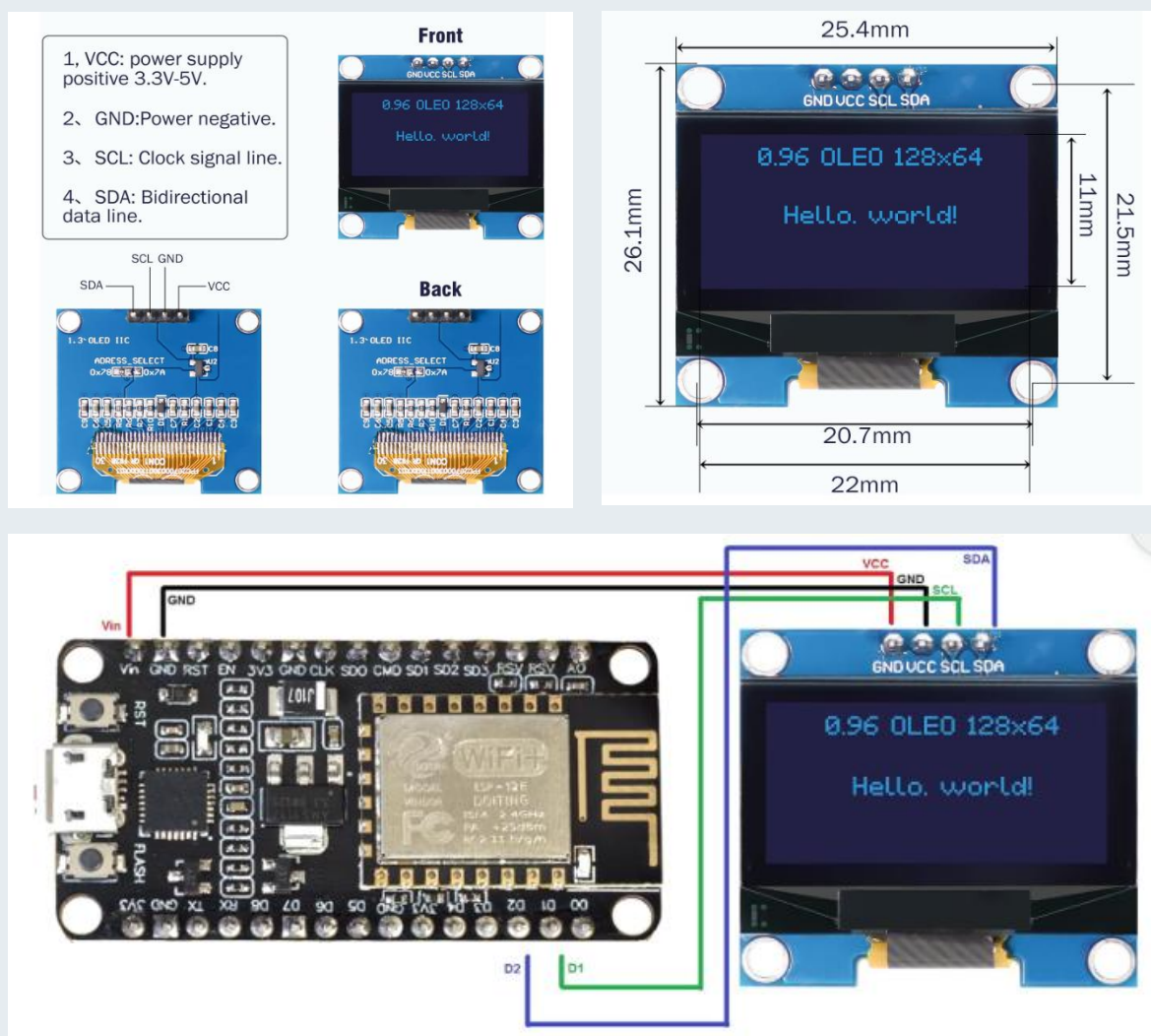
L'unità centrale del sistema è l'**Arduino ESP32 Nano**, un microcontrollore progettato per l'integrazione di sistemi IoT. Per sua natura, un microcontrollore è un circuito integrato dedicato all'esecuzione di compiti specifici in tempo reale. In questo contesto, l'ESP32 agisce come il coordinatore dei flussi di informazione. Basato sul chip **ESP32-S3** con architettura Dual-Core a 240 MHz, il dispositivo garantisce la potenza di calcolo necessaria per mantenere una connessione Wi-Fi stabile verso il server remoto mentre gestisce, in parallelo, la ricezione dei comandi dal pulsante fisico e l'invio delle stringhe di dati alla stampante termica. L'adozione dello standard **USB-C** assicura un'interfaccia moderna per



l'alimentazione e la programmazione tramite **Arduino IDE**, facilitando la prototipazione rapida. Dal punto di vista elettrico, sebbene la scheda possa essere alimentata a 5V, l'intera logica di funzionamento e la comunicazione sui pin GPIO avvengono a **3.3V**, un parametro critico per garantire l'integrità dei componenti elettronici interfacciati.

3.2.2 Modulo Display OLED SSD1306.

Il modulo **OLED SSD1306** è un display compatto e a basso costo, progettato per visualizzare informazioni in formato alfa-numerico. Comunica tramite **I²C (Inter-Integrated Circuit)**, un protocollo seriale che utilizza solo due linee, SDA (dati) e SCL (clock), permettendo al microcontrollore di inviare comandi e aggiornare il display in modo semplice ed efficiente. Nel contesto del D-Line, il modulo viene utilizzato per mostrare **il numero attuale di persone in fila**. Ogni volta che viene premuto il pulsante collegato, il conteggio viene incrementato di 1 entro pochi secondi, rimanendo sincronizzato con il server e il database centrale. La scelta di questo componente è stata dettata da diversi fattori: le **dimensioni compatte**, il **prezzo contenuto** e la **facilità di installazione**. Queste caratteristiche lo rendono ideale per integrare un feedback visibile e immediato nel sistema, senza complicare il progetto o aumentare eccessivamente i costi. Prende l'alimentazione dall'esp32. Dal punto di vista elettrico, il display è alimentato direttamente dai pin dell'**ESP32**, soluzione che semplifica l'architettura circuitale ed elimina la necessità di regolatori di tensione dedicati per questa periferica.



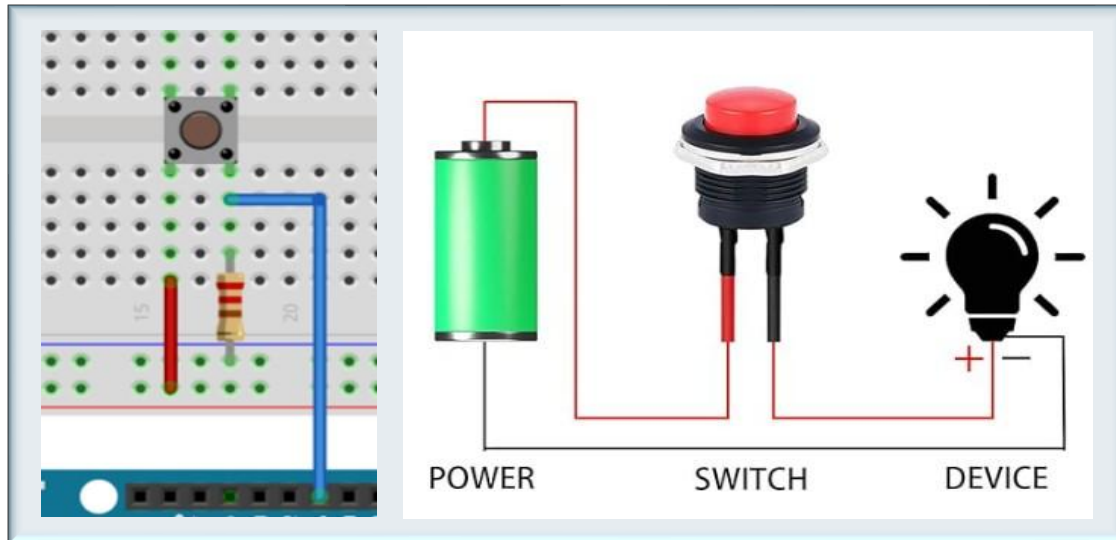
3.2.3 Stampante Termica EM5822

L'integrazione della stampante termica **EM5822** costituisce il terminale di output fisico del sistema, incaricato di trasformare istantaneamente i dati digitali in ticket cartacei. Questa periferica sfrutta la tecnologia termica diretta, che genera testo e grafica applicando calore su una speciale carta termosensibile, eliminando completamente la necessità di inchiostri o nastri e riducendo drasticamente i costi di manutenzione e i consumi d'esercizio. Sotto il profilo elettrico, il modulo opera in un range compreso tra 5V e 9V e richiede obbligatoriamente un alimentatore esterno dedicato, poiché i picchi di assorbimento durante la fase di stampa possono superare i 2A. Questa soglia energetica risulterebbe insostenibile per il microcontrollore, che subirebbe cali di tensione critici o danni permanenti ai circuiti interni se dovesse alimentare direttamente la testina termica. La comunicazione logica è affidata alla tecnologia **TTL (Transistor-Transistor Logic)**, uno standard di trasmissione seriale che utilizza livelli di tensione definiti per rappresentare i segnali binari. In questo protocollo, le informazioni viaggiano sotto forma di impulsi elettrici interpretati direttamente dai pin TX (trasmissione) e RX (ricezione), permettendo uno scambio di dati rapido e privo di interfacce di conversione pesanti. Un aspetto tecnico fondamentale per l'integrità del segnale risiede nel cablaggio a cinque fili, dove la condivisione della linea di massa (GND) tra la stampante e il microcontrollore funge da riferimento elettrico comune. La compattezza del modulo, unita all'economicità dei materiali e alla stabilità del collegamento seriale, rende questa soluzione ottimale per un'integrazione hardware efficiente e professionale.



3.2.4 Pulsante

Il pulsante funge da interfaccia di input principale per l'attivazione della richiesta del ticket. Per garantire una lettura corretta del segnale da parte del microcontrollore, è stato configurato in modalità **Pull-Down**. In questo schema, un terminale del pulsante è collegato alla linea di alimentazione positiva (3.3V), mentre l'altro terminale è connesso contemporaneamente a un pin logico (GPIO) dell'ESP32 e alla massa (GND) tramite una resistenza da **10Kohm**. Quest'ultima è fondamentale per mantenere il pin logico in uno stato "basso" (LOW) stabile quando il pulsante non è premuto, evitando che interferenze elettriche causino falsi trigger.



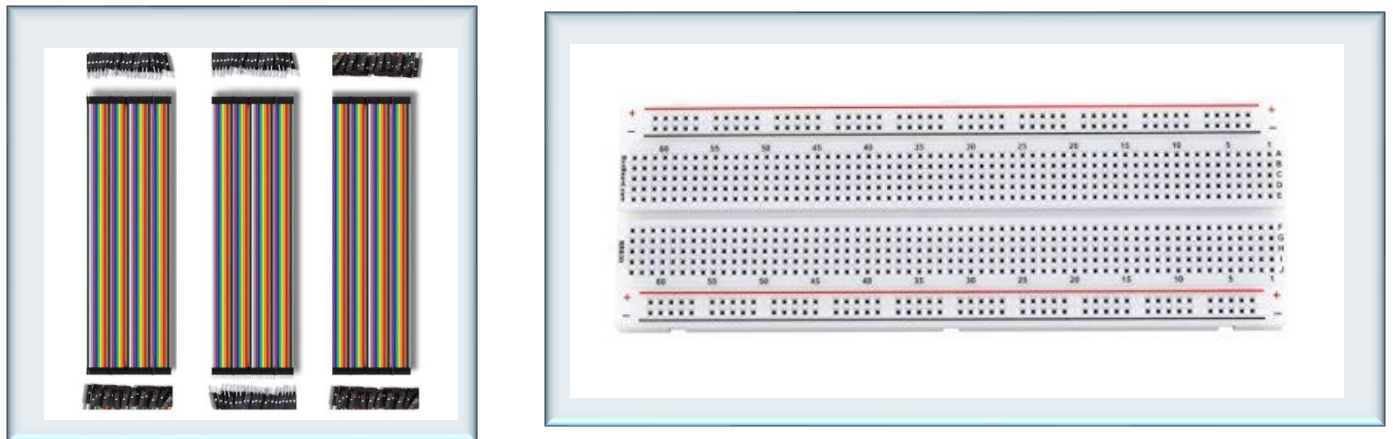
3.2.5 Sistema di Alimentazione

L'alimentazione della stampante termica è affidata a un adattatore CA/CC universale, selezionato per la sua capacità di erogare una corrente massima di **3A** con una tensione d'uscita regolabile fino a **12V** (configurata nel range 5V-9V per i requisiti del modulo). L'impiego di questa unità esterna è fondamentale per isolare il carico energetico della testina termica dal microcontrollore, garantendo lo spunto di potenza necessario durante l'emissione dei ticket senza causare cadute di tensione sull'ESP32.

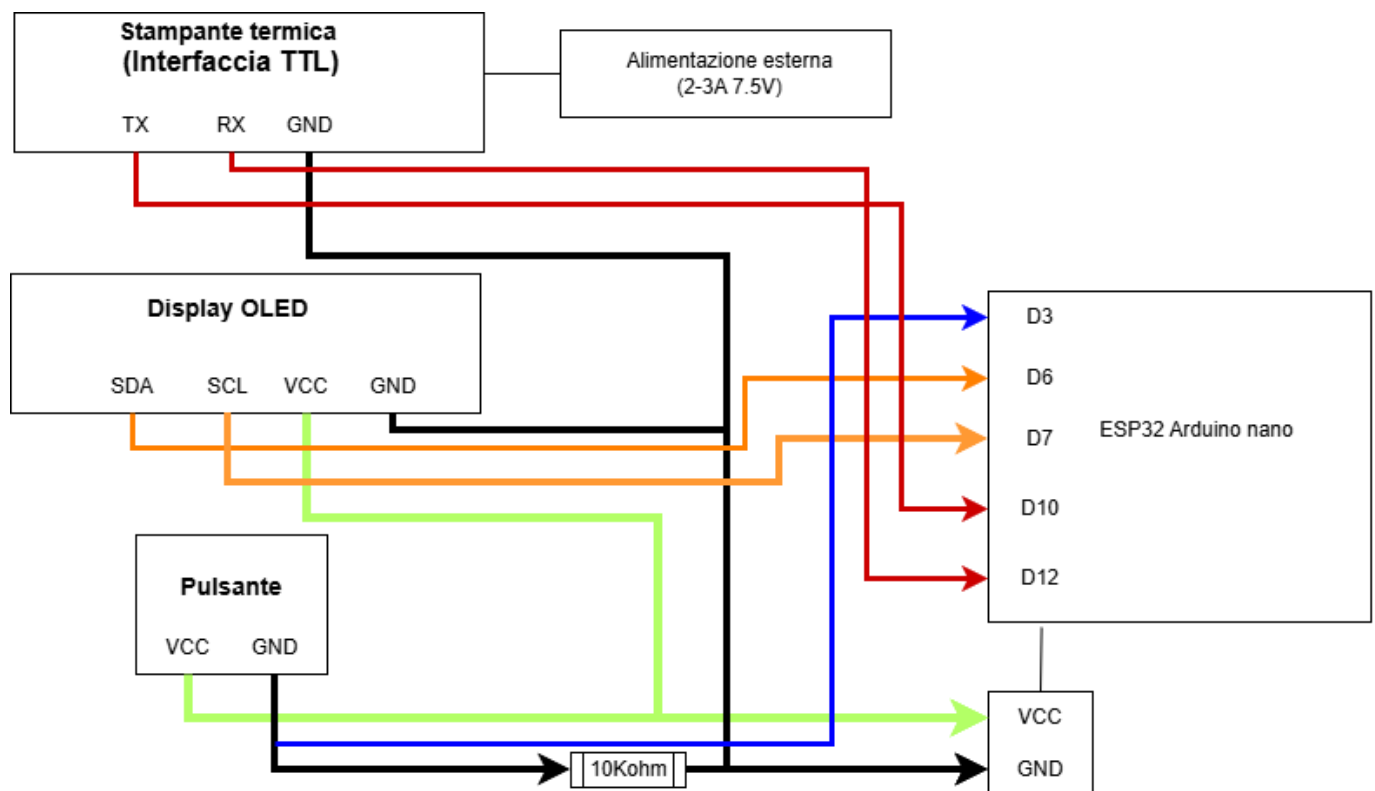


3.2.6 Breadboard & cavi

La **breadboard** è una basetta di prototipazione utilizzata per interconnettere i componenti all'ESP32 in modo rapido e senza ricorrere a saldature. La sua funzione principale è semplificare l'organizzazione del circuito, facilitando la condivisione delle linee di massa (**GND**) e di alimentazione (**VCC**) provenienti dal microcontrollore. Attraverso i bus laterali, la basetta permette di distribuire stabilmente il potenziale elettrico a tutti i moduli del sistema, come il display e il pulsante, garantendo che ogni periferica riceva il riferimento di tensione necessario per il corretto funzionamento della logica digitale. Il cablaggio è completato dall'utilizzo di cavi jumper compatibili con l'ecosistema Arduino, che agevolano le connessioni tra le varie componenti, mentre l'impiego di nastro isolante assicura la stabilità finale dei contatti e la protezione dell'intera struttura.



3.3 Schema di Collegamento



4. Sviluppo Software

4.1 Panoramica

In questa sezione viene esplorato il nucleo logico e la componente "intelligente" del sistema **DLine**. Vengono descritti nel dettaglio i meccanismi di interazione effettiva tra il Client e il Server, analizzando le dinamiche di rete e i protocolli di comunicazione impiegati per garantire la sincronizzazione in tempo reale dei dati tra l'hardware fisico e le interfacce web.

4.2 Ambienti di Sviluppo e Testing Locale

Data la natura ibrida del progetto, che unisce l'elettronica IoT allo sviluppo Web, la stesura del codice ha richiesto l'impiego di ambienti di sviluppo (IDE) separati e di un software per l'emulazione server locale. Le tecnologie adottate per questa fase preparatoria sono:

- **Arduino IDE:** Utilizzato esclusivamente per la programmazione del microcontrollore ESP32. Questo ambiente ha permesso la stesura del firmware in linguaggio C++, l'inclusione delle librerie necessarie per gestire display e stampante, e la successiva compilazione e caricamento (*flashing*) del codice sulla memoria della scheda. La scelta di questo IDE si è rivelata fondamentale per la sua versatilità gestionale tramite il *Board Manager*, che ha garantito l'installazione rapida di driver e dipendenze specifiche a seconda del fornitore del chip hardware (come ad esempio le varianti *AZ-Delivery*).
- **Visual Studio Code (VS Code):** Scelto come editor principale per lo sviluppo dell'intera infrastruttura web. Grazie alla sua leggerezza e al supporto avanzato per i linguaggi web e i formati di interscambio dati (PHP, HTML, CSS, JavaScript e JSON), ha ottimizzato la scrittura, l'organizzazione dei file *server-side* e l'implementazione delle interfacce utente.
- **XAMPP (Ambiente di Test):** Prima di eseguire il deployment (messa in produzione) sul servizio di Web Hosting remoto, l'ecosistema logico del progetto è stato collaudato in locale. L'utilizzo di XAMPP, un pacchetto software multiplatforma, ha permesso di emulare un server web completo per verificare il corretto instradamento dei pacchetti dati, la stabilità delle API e l'integrità delle logiche del database prima dell'esposizione sulla rete pubblica.

4.3 Protocolli di Comunicazione e Tecnologie Web

Per garantire una comunicazione sicura, veloce e compatibile con l'architettura di web hosting, il sistema si affida allo standard **HTTPS**. Lo scambio di informazioni tra l'hardware (DLine) e il server centrale avviene tramite richieste **HTTP** di tipo GET e POST. Al fine di ottimizzare la banda e facilitare l'elaborazione lato client, i dati strutturati vengono scambiati utilizzando il formato leggero **JSON** (JavaScript Object Notation).

4.3 Logica e Funzionamento del Firmware

Il codice in C++ caricato sull'ESP32 è strutturato per operare in un ciclo continuo (polling) che gestisce in parallelo gli input fisici e le chiamate di rete. Al momento dell'avvio, il microcontrollore inizializza la connessione Wi-Fi, il display OLED (tramite bus I2C) e la stampante termica (tramite interfaccia Seriale ed ESC/POS), scaricando dal server il valore iniziale della coda.

Il cuore logico del firmware risiede nella gestione del pulsante fisico: per evitare letture multiple involontarie dovute all'azione meccanica del tasto, è stato implementato un sistema di "debounce" software. Il sistema rileva esclusivamente la transizione di stato da "non premuto" a "premuto" e applica un ritardo di sicurezza. A seguito di una pressione valida, l'ESP32 innesca una sequenza di eventi:

1. Invia una richiesta di incremento all'API esposta dal server.
2. Interroga nuovamente il server per ottenere il nuovo totale aggiornato, effettuando il parsing del payload JSON ricevuto in risposta.
3. Aggiorna l'interfaccia grafica del display OLED.
4. Invia i comandi di stampa per l'emissione fisica del ticket.

4.4 Persistenza dei Dati (Database MySQL)

La memorizzazione dello stato attuale della coda è affidata a un **database relazionale MySQL** ospitato sul server. L'architettura dei dati è volutamente minimalista ed efficiente: si basa su un'unica tabella denominata **dline_ticket** che conserva un singolo record (identificato da ID=1).

I campi principali soggetti ad aggiornamento sono due valori interi: il numero (che rappresenta il ticket attualmente servito dall'operatore) e il totale (che indica il numero progressivo di biglietti staccati dal dispositivo fisico). Questo approccio centralizzato assicura che lo stato della coda rimanga invariato e protetto anche in caso di improvvisa disconnessione o spegnimento del dispositivo hardware locale. L'accesso in scrittura e lettura al database è consentito esclusivamente agli script PHP (driver MySQLi), garantendo sicurezza e integrità.

4.5 Integrazione API e Sincronizzazione di Rete

Il dialogo continuo tra le interfacce web (sia l'Area Utente che il Pannello Admin) e il server cloud è gestito tramite tecniche di aggiornamento asincrono. Al primo accesso, le pagine sfruttano il Server-Side Rendering (SSR) tramite PHP per fornire immediatamente la struttura HTML popolata con i dati estratti dal database.

Successivamente, entra in gioco la logica **AJAX** (Asynchronous JavaScript and XML), implementata attraverso la moderna funzione JavaScript `fetch()`. Questo meccanismo permette ai client web di interrogare l'endpoint `api_server.php` a intervalli regolari (ogni 5 secondi) lavorando in background. Il server risponde restituendo un oggetto formattato in **JSON** contenente lo stato aggiornato dei contatori. Il JavaScript elabora questo payload e aggiorna dinamicamente i nodi del DOM (Document Object Model) a schermo. Grazie a questa implementazione AJAX, l'interfaccia si mantiene sincronizzata in tempo reale con l'hardware DLine e con le azioni dell'amministratore, offrendo un'esperienza utente fluida e senza la necessità di ricaricare l'intera pagina web. Nell'Area Admin, inoltre, la manipolazione della coda (avanti, indietro, reset) è affidata a richieste HTTP POST tradizionali, i cui effetti vengono immediatamente riflessi nel ciclo di fetch successivo di tutti i dispositivi connessi.

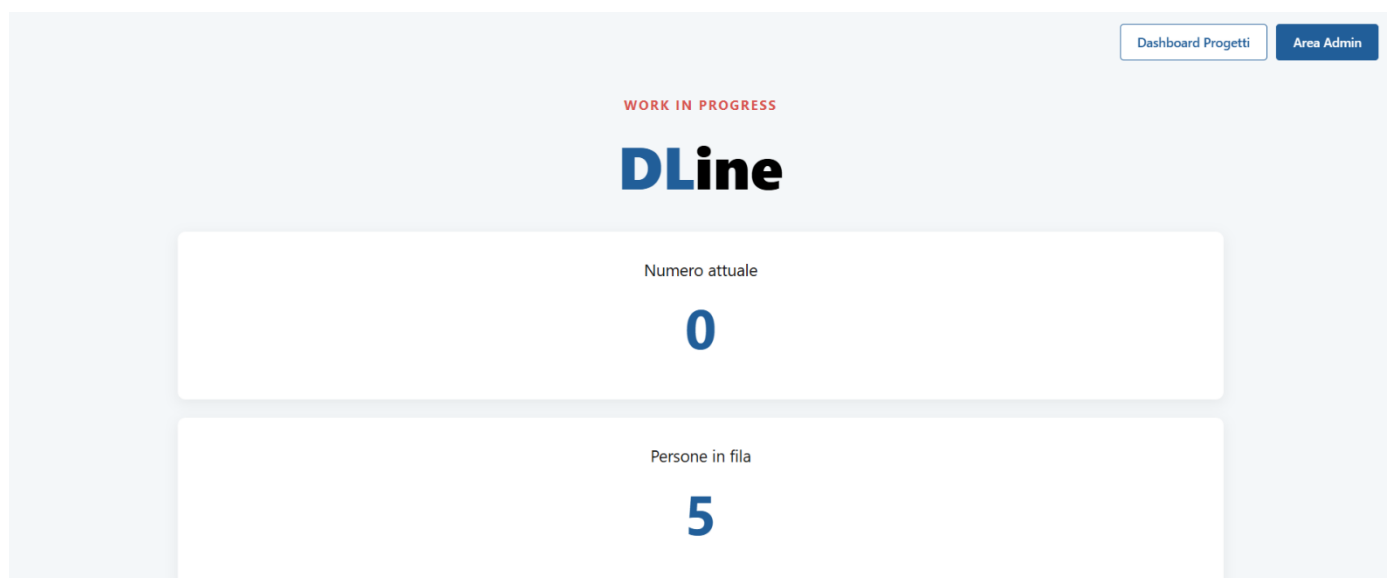
5. Interfaccia (Mockup)

Questo capitolo è dedicato allo studio dell'interfaccia utente (UI) sia per la componente digitale (le pagine web) che per quella fisica (il dispositivo sul bancone). L'obiettivo principale è garantire un'interazione intuitiva, rapida e priva di attriti per tutte le tipologie di utenti coinvolti.

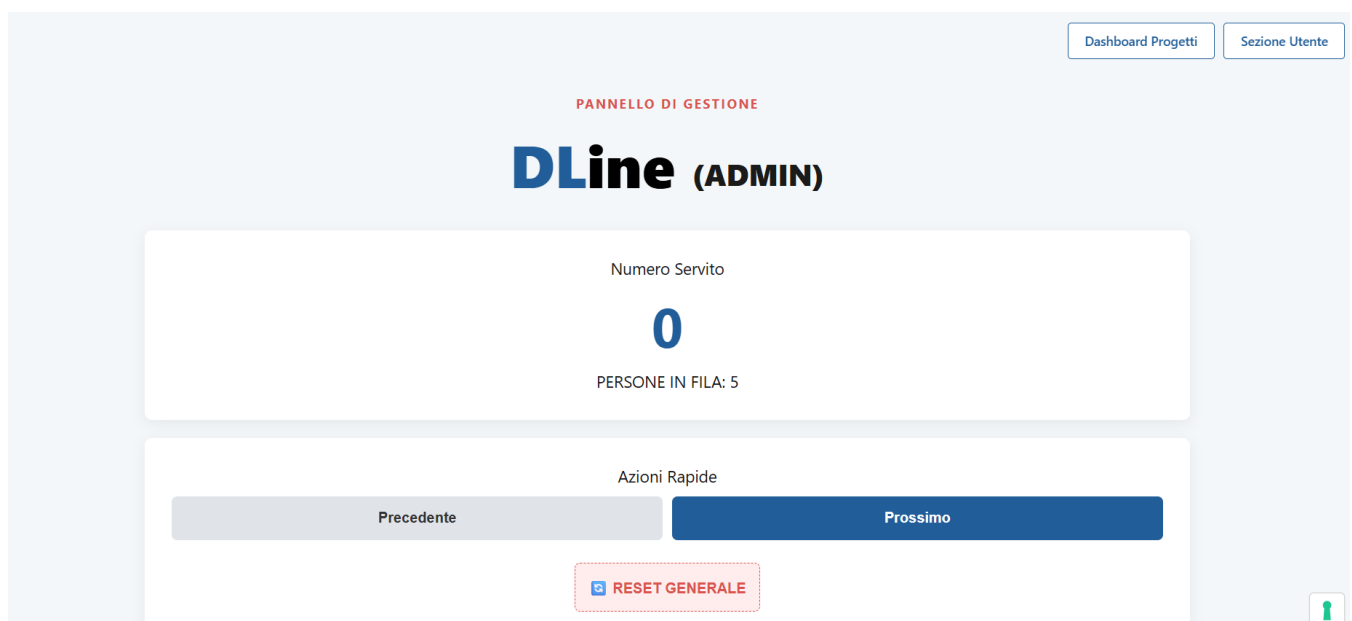
5.1 Interfacce Web: Frontend Pubblico e Admin

Il design delle interfacce digitali è stato sviluppato tenendo conto dei diversi contesti di fruizione visiva e operativa:

- **Monitor Pubblico:** Progettata per essere visualizzata su schermi distanti (es. una Smart TV affissa nel locale). Utilizza un layout minimalista, font ad alta leggibilità (sans-serif) e un forte contrasto cromatico. L'architettura visiva dà massima priorità gerarchica al "Numero Attuale" in fase di servizio, affiancato in formato ridotto dal totale delle persone in attesa.



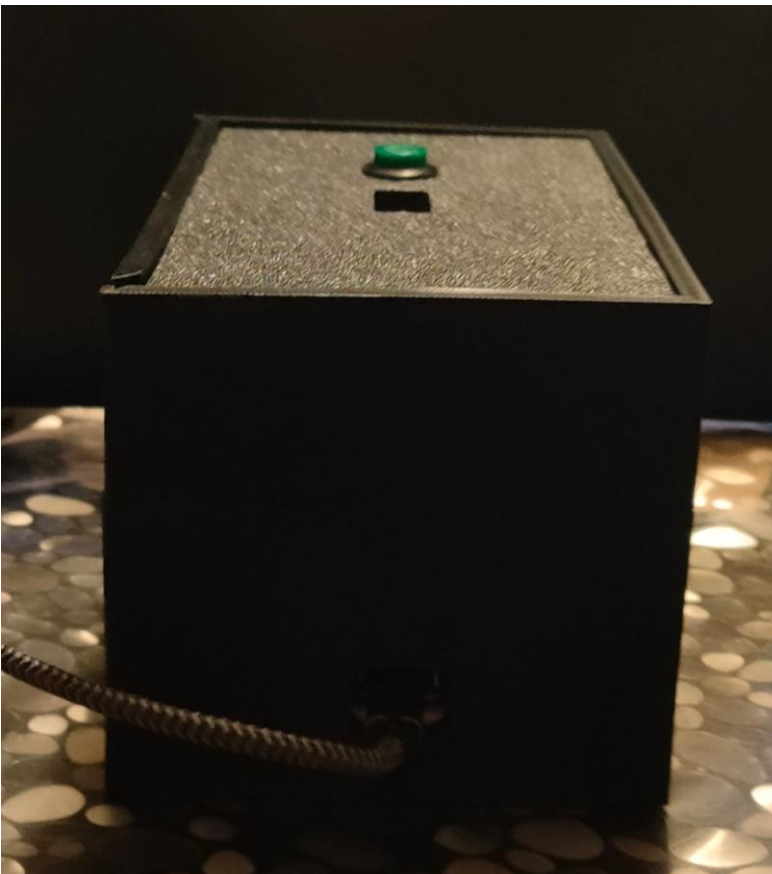
- **Dashboard Admin:** L'interfaccia riservata al personale è ottimizzata per l'uso rapido. I pulsanti operativi (Avanti/Reset) sono ampi e ben distanziati per evitare tocchi accidentali, risultando ergonomici anche su dispositivi touch-screen durante le fasi di lavoro più concitate.



5.2 Progettazione del Contenitore Fisico (Case Design)

L'usabilità e il design del sistema DLine passano inevitabilmente anche per il suo hardware. Il contenitore fisico è stato ingegnerizzato per racchiudere in modo compatto il microcontrollore, i cablaggi e la stampante termica, garantendo stabilità sul bancone di lavoro.

- **Scelte Ergonomiche:** Il layout esterno è stato studiato per guidare la mano dell'utente. Il pulsante di input è posizionato sulla plancia superiore per risultare immediatamente accessibile, mentre la fessura per l'uscita dello scontrino è calibrata per garantire uno strappo netto e agevole del ticket. Il display OLED è stato posizionato con un'inclinazione frontale per favorire una lettura immediata dei dati locali.



6. Analisi Economica

6.1 Distinta Base (BOM)

La **Bill of Materials (BOM)**, o Distinta Base, rappresenta l'inventario completo e dettagliato di tutti i componenti hardware necessari per la realizzazione di una singola unità del dispositivo **DLine**. Questa sezione permette di analizzare l'incidenza economica di ogni modulo sul costo di produzione finale del prototipo. I costi riportati di seguito fanno riferimento ai prezzi di mercato al dettaglio; molti componenti sono stati acquistati in kit per ottimizzare i costi logistici, ma il calcolo del totale si basa sulla quota parte utilizzata per l'assemblaggio di un singolo dispositivo.

Componente	Quantità	Prezzo unitario	Prezzo totale	Note
Arduino ESP32 Nano	1	21,60	21,60	-
Stampante termica	1	29,68	29,68	-
Pulsante	12	0,75	8,99	Acquistato kit da 12 pezzi
Modulo display	3	4,99	14,99	Acquistato kit da 3 pezzi
Breadboard + cavi	4 + 3	5,33	15,99	Acquistato un set di 4 breadboard e 3 kit di cavi
Contenitore	1	6,11	6,11	-

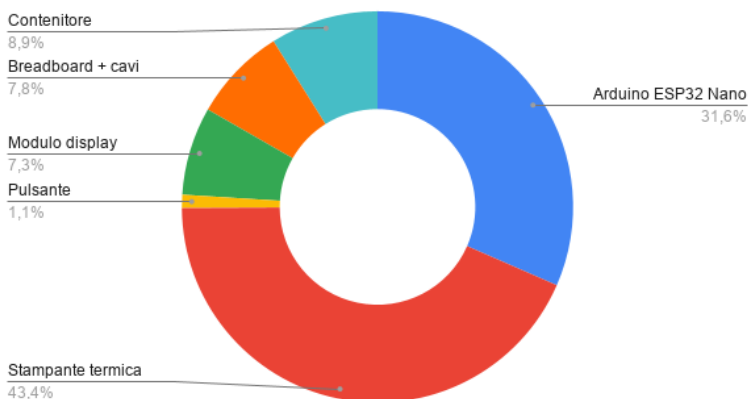
6.2 Costi infrastrutturali

Il mantenimento dell'architettura digitale richiede l'utilizzo di un servizio di **Web Hosting** per la gestione del dominio, della logica di backend e del database. La scelta del provider dipende dalle specifiche esigenze di traffico e dalla stabilità richiesta; per questo prototipo si è scelto di utilizzare la piattaforma **Altervista**. Il canone per il mantenimento del dominio e l'allocazione delle risorse necessarie è di circa **25,00 € annui**.

6.2 Valutazione totale

L'analisi economica del progetto si divide tra investimento iniziale e costi di gestione ricorrenti. Il costo totale per la realizzazione della componente hardware, derivante dalla sommatoria dei singoli moduli utilizzati, ammonta a **68,46 €**. A questa spesa una tantum si aggiunge il costo operativo dell'infrastruttura di rete, pari a un canone annuale di **25,00 €**.

Prezzo unitario rispetto a Componente



7. Conclusioni e Sviluppi Futuri

7.1 Trade Off

Trade Off	Descrizione
Costo vs. Robustezza Industriale	Si è scelto di utilizzare componenti basate sull'ecosistema Open Hardware (Arduino/ESP32) anziché PLC o hardware industriale proprietario. Sebbene i sistemi industriali offrano una protezione fisica superiore, l'uso di microcontrollori commerciali ha permesso di ridurre i costi di produzione di circa l'80%, mantenendo prestazioni operative identiche per contesti commerciali standard.
Connettività Wi-Fi vs. Cablaggio Ethernet	L'integrazione del modulo Wi-Fi dell'ESP32 è stata preferita alla connessione cablata (Ethernet). Questo trade-off sacrifica una minima stabilità di segnale in presenza di forti interferenze a favore di una facilità di installazione estrema, eliminando la necessità di cablaggi strutturati all'interno dei locali.
Stampa Termica vs. Digitale Pura	Nonostante la tendenza alla dematerializzazione, si è deciso di mantenere l' output cartaceo . Questo compromesso tra innovazione digitale e abitudine dell'utente garantisce l'accessibilità del servizio anche a fasce di utenza non digitalizzate, evitando che l'innovazione tecnologica diventi una barriera all'ingresso.
Web Hosting Condiviso vs. Server Dedicato	L'adozione di una piattaforma come Altervista rappresenta un trade-off tra controllo totale dell'infrastruttura e semplicità di gestione. Sebbene un server dedicato offra maggiore personalizzazione, l'hosting condiviso permette di abbattere i costi di manutenzione e configurazione, garantendo al contempo una scalabilità immediata e una gestione semplificata del database MySQL.
Alimentazione Esterna vs. Bus del Microcontrollore	La decisione di utilizzare un alimentatore esterno dedicato per la stampante, invece di alimentare tutto tramite il microcontrollore, aumenta leggermente l'ingombro dei cablaggi. Tuttavia, questo compromesso è indispensabile per garantire la stabilità del sistema , proteggendo l'elettronica sensibile dai picchi di corrente richiesti dalla testina termica durante la stampa.

7.1 Risultati Ottenuti

Il collaudo del prototipo DLine ha confermato il pieno raggiungimento degli obiettivi prefissati in fase di progettazione. Il sistema supera con successo le limitazioni dei classici dispositivi eliminacode standalone, dimostrando la fattibilità e l'efficienza di un'architettura IoT cloud-native. Nello specifico, la comunicazione bidirezionale tra il microcontrollore ESP32 e il Web Server tramite richieste HTTPS e payload JSON si è rivelata stabile e reattiva. L'uso delle chiamate asincrone (Fetch API) garantisce che sia l'hardware fisico sia le interfacce web riflettano il medesimo stato della coda in tempo reale, senza conflitti di concorrenza sul database. A completamento e dimostrazione del corretto deployment dell'infrastruttura software, le interfacce operative (Dashboard Utente e Pannello Admin) sono attualmente accessibili sul mio sito web personale, all'interno della sezione dedicata ai "Progetti".

7.2 Possibili Implementazioni

L'architettura scalabile e modulare su cui si fonda il progetto pone solide basi per future espansioni. Sfruttando la centralizzazione dei dati su Web Hosting, tra i prossimi sviluppi mirati a migliorare l'esperienza utente e l'utilità commerciale del sistema si prevedono:

- **Prenotazione e Monitoraggio tramite QR Code:** Introduzione di una funzionalità che permetta ai clienti di scansionare un QR Code (esposto in vetrina o stampato dalla termica) per visualizzare lo stato della coda sul proprio smartphone. Questo consentirebbe agli utenti di

allontanarsi dal punto vendita e monitorare l'avanzamento dei numeri da remoto, riducendo gli assembramenti fisici.

- **Gestione Multi-Coda (Servizi Differenziati):** Espansione dell'infrastruttura logica e del database per supportare la gestione simultanea di code separate in base al servizio richiesto (ad esempio "Cassa", "Resi", o "Informazioni"). Questa implementazione richiederà un aggiornamento del dispositivo hardware Client, per consentire la selezione preventiva del reparto, e un adeguamento delle API PHP per instradare l'incremento numerico su record distinti all'interno del database.

8. Glossario

- **API (Application Programming Interface):** Insieme di procedure, protocolli e strumenti che permettono a due applicazioni software diverse di comunicare e interagire tra loro, scambiando dati e funzionalità in modo standardizzato e indipendente dalle rispettive implementazioni interne.
- **Client-Server:** Modello di architettura informatica di rete in cui le risorse o i servizi sono distribuiti tra i fornitori di tali risorse, detti *Server*, e i richiedenti, detti *Client*. Il Client invia una richiesta tramite protocolli di rete, e il Server la elabora fornendo la risposta adeguata.
- **DBMS (Database Management System) e SQL (Structured Query Language):** Il DBMS è un sistema software progettato per consentire la creazione, la manipolazione e l'interrogazione efficiente di database relazionali, garantendo la sicurezza e la persistenza dei dati. SQL è il linguaggio standardizzato utilizzato per interfacciarsi con il DBMS ed eseguire operazioni di lettura, inserimento, aggiornamento e cancellazione.
- **ESC/POS (Epson Standard Code for Point of Sale):** Linguaggio di comando universale standardizzato, composto da sequenze esadecimali o byte, utilizzato per il controllo delle stampanti termiche e di ricevute. Permette di gestire da codice la formattazione del testo, l'inserimento di grafica o codici a barre e operazioni meccaniche come il taglio o l'avanzamento della carta.
- **Fetch API, AJAX e Richiesta Asincrona:** Tecniche di programmazione web che consentono a una pagina HTML di scambiare dati con un server in background. Questo paradigma permette di aggiornare dinamicamente porzioni specifiche dell'interfaccia utente (DOM) in risposta a nuovi dati, senza la necessità di ricaricare interamente la pagina web.
- **Firmware:** Software di basso livello integrato e caricato in una memoria non volatile di un dispositivo hardware (come un microcontrollore o una ROM). Il suo scopo è fornire le istruzioni basilari per controllare l'elettronica del dispositivo e permetterne l'interazione con altro hardware o software di livello superiore.
- **I²C (Inter-Integrated Circuit) e UART (Universal Asynchronous Receiver-Transmitter):** Standard e protocolli di comunicazione seriale utilizzati in elettronica. L'I²C è un bus di comunicazione sincrono a due fili (SDA per i dati, SCL per il clock) ideale per collegare più periferiche su brevi distanze. L'UART è un protocollo asincrono punto-a-punto che utilizza due linee separate per la trasmissione (TX) e la ricezione (RX) dei dati logici.
- **IDE (Integrated Development Environment):** Ambiente di sviluppo integrato. È uno strumento software che fornisce ai programmatori un ambiente completo per la scrittura del codice. Solitamente include in un'unica interfaccia un editor di testo avanzato, strumenti di build/compilazione, librerie e un debugger, semplificando e velocizzando il ciclo di sviluppo del software.
- **IoT (Internet of Things):** Paradigma tecnologico che descrive la rete di oggetti fisici ("cose") dotati di sensori, capacità di elaborazione, software e tecnologie di connettività, allo scopo di

connettersi e scambiare dati in tempo reale con altri dispositivi e sistemi informatici tramite Internet.

- **JSON (JavaScript Object Notation):** Formato di file e standard di scambio dati aperto e leggero. Utilizza testo leggibile dall'uomo per trasmettere oggetti dati strutturati mediante coppie "chiave-valore" e array. Grazie alla sua sintassi indipendente dal linguaggio, è ampiamente utilizzato per la comunicazione client-server sul web.
- **Protocollo HTTP (HyperText Transfer Protocol) - Metodi GET e POST:** HTTP è il protocollo a livello di applicazione fondamentale per il trasferimento di dati sul World Wide Web. Il metodo *GET* viene utilizzato esclusivamente per richiedere e leggere dati da una risorsa specifica; il metodo *POST* viene impiegato per inviare dati al server affinché vengano elaborati o archiviati, incapsulando in modo sicuro il payload nel corpo della richiesta.
- **VCC e GND:** In ingegneria elettronica, *VCC* indica il potenziale positivo dell'alimentazione elettrica di un circuito integrato (la tensione di alimentazione principale). *GND* (Ground o massa) rappresenta il potenziale di riferimento zero (0V) del circuito, fondamentale per la chiusura dei circuiti elettrici e come punto di riferimento comune per i segnali logici.
- **Web Hosting:** Servizio informatico che consiste nell'allocare, su un server connesso a Internet, una porzione di spazio fisico o virtuale e le relative risorse computazionali per ospitare le pagine di un sito web, un'applicazione o un database, rendendoli pubblicamente accessibili e navigabili attraverso la rete.